HW4: Implement CTDE method in MPE simple spread environment

Shui Jie, 2401112104

May 13, 2025

1 Introduction

This report details the implementation and evaluation of QMIX, a multi-agent reinforcement learning algorithm, for the Multi-Agent Particle Environment (MPE) simple spread scenario. QMIX was selected for its balanced approach to implementation complexity and performance potential, leveraging monotonic value function factorization to represent a more expressive class of joint action-value functions compared to simpler algorithms like VDN. The implementation closely follows the architecture proposed in the original paper while adjusting network parameters to accommodate computational constraints.

Through multiple training iterations, this work explores various hyperparameter configurations and their effects on agent behavior and learning stability. Despite maintaining the monotonicity constraint throughout training (with correlations reaching 0.9), challenges emerged including policy collapse, unstable rewards, and agents learning counterproductive behaviors. Analysis revealed potential limitations in the reward function design, leading to a proposed modified reward structure that better captures task requirements by explicitly rewarding landmark coverage and penalizing agent collisions. This implementation demonstrates both the theoretical strengths of QMIX's architecture and the practical challenges of applying it to cooperative multi-agent tasks with sparse rewards.

2 Selection of Algorithms

2.1 Centralized Training with Decentralized Execution

The foundation of Centralized Training with Decentralized Execution (CTDE) algorithms lies in their ability to leverage global information during the training phase while ensuring agents can act independently using only local observations during deployment. This approach strikes an optimal balance between learning efficiency and practical applicability. In this implementation, I collect complete trajectories of all agents during training, incorporating their individual accessible states and actions. However, during execution, each agent's policy relies exclusively on locally observable information, maintaining the decentralized execution principle essential for real-world deployment.

2.2 QMIX Framework and Monotonicity

QMIX distinguishes itself through monotonic value function factorization, which enables it to represent a substantially larger class of joint action-value functions compared to simpler alternatives like Value Decomposition Networks (VDN). The key innovation of QMIX is its enforcement of the monotonicity constraint:

$$\frac{dQ\left(s,a\right)}{dQ_i\left(s,a\right)} \geq 0, \forall i \tag{1}$$

This constraint is implemented through neural networks with weights restricted to non-negative values, allowing the algorithm to approximate any monotonic function with arbitrary precision. Importantly, QMIX

permits the factorization of $Q(s, a)$ to vary across different states, as the network weights are dynamically generated rather than fixed, providing additional flexibility in capturing state-dependent coordination patterns.

2.3 Comparative Analysis: QMIX vs. VDN

When compared to VDN, QMIX offers significantly greater expressive power. VDN constrains itself to a simple additive relationship between individual and joint action-values:

$$Q(s, a) = \sum Q_i(s, a) \tag{2}$$

In contrast, QMIX employs a more flexible monotonic relationship:

$$Q(s, a) = f(Q_1(s, a), Q_2(s, a), Q_3(s, a)) \tag{3}$$

where f represents a learnable monotonic mixing function. This enhanced representation capability allows QMIX's monotonic mixing network to capture the complex coordination patterns necessary for effective landmark coverage in the MPE simple spread scenario, while still maintaining the critical monotonicity constraint that guarantees consistency between local and global optima.

2.4 Comparative Analysis: QMIX vs. COMA

QMIX also demonstrates advantages over Counterfactual Multi-Agent Policy Gradients (COMA) in terms of sample efficiency. COMA relies on counterfactual policy gradients, which can suffer from high variance during training, potentially requiring more samples to converge reliably. Conversely, QMIX's value-based approach tends to achieve greater sample efficiency, particularly in environments with discrete action spaces. Given that the MPE simple spread scenario features a discrete action space with five possible actions per agent (no action, move left, move right, move down, move up), QMIX represents a particularly appropriate algorithmic choice for this task.

3 Network Architecture

3.1 Original Paper Design and Parameters

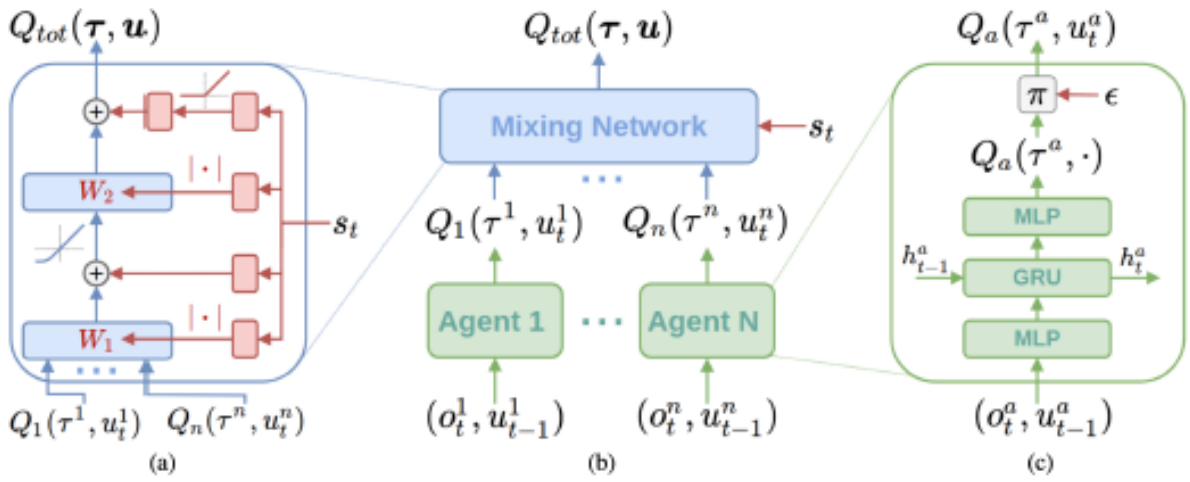


Figure 1: QMIX design: Left (blue): The mixing network architecture, Middle: The overall agent structure, Right (green): The hypernetwork that generates parameters for the mixing network.

The original pipeline is composed of two neural networks: an individual policy network and a mixing network. Each agent is controlled by a policy network that processes the current local observation and the previous action to generate estimated Q-values for all possible actions that can be taken. An ϵ-greedy policy is then used to select an action based on these Q-values.

The mixing network integrates all the Q-values generated by each agent along with the global observation of the current state to output a Q_{total} value, which represents the expected return for the entire team when taking joint action a in state s.

3.2 My Neural Network Architecture

I adhered closely to the overall architecture described in the original QMIX paper. However, due to computational resource limitations, I adjusted the parameters and neural network sizes to facilitate practical training. Specifically, I increased the hidden state dimension for my networks to 128 instead of the 64 indicated in the paper.

3.2.1 AgentNetwork

The agent network consists of three layers: two TensorFlow dense layers sandwiching a TensorFlow GRU layer. The model maintains a hidden state as input for the GRU layer during single episode data collection.

The network architecture can be summarized as follows:

- **Input dimension**: 19 (local observation: 18 + previous action: 1)
- **Hidden layer dimension**: 128
- **Output dimension**: 5 (corresponding to each possible action)
- **Action selection**: ϵ-greedy policy based on output Q-values

3.2.2 MixNetwork

The mixing network combines the individual agents' Q-values into a joint action-value while preserving monotonicity. The architecture implements two key components:

1. **Weight generation network**: Two layers of TensorFlow dense layers that process the global observation (dimension 54) to produce weights for the actual mixing layers. These weights are passed through an absolute value operation to ensure non-negativity, guaranteeing the monotonicity constraint.
2. **Bias generation network**: Two additional layers of TensorFlow dense layers that also take the global observation (dimension 54) as input to generate biases for the actual layers. The first bias values remain unchanged, while the second bias values are passed through a ReLU activation function.

The complete mixing network accepts two separate inputs:

- Global state (dimension 54) to generate weights and biases
- Concatenated Q-values from all agent networks (dimension 3, one per agent)

The network outputs a single value representing Q_{total}, which is used for centralized training while maintaining the ability for decentralized execution during deployment.

4 Training Iterations

4.1 Original Paper Parameters

The original QMIX paper established the following hyperparameters for training:

1. Learning rate: 5×10^{-4}
2. Update interval: Once every 200 episodes collected
3. Buffer size: Maintains most recent 4000 episodes
4. Agent epsilon during data collection: Starts with 1, gradually decreasing to 0.05 over 50,000 timesteps
5. Batch size: 32

4.2 Loss Function

I followed the methodology described in the paper to implement the loss function, which is computed as the sum of temporal difference errors using the Q_{total} value. During each update, a random batch of complete episodes is selected from the replay buffer and used to calculate the loss.

The process follows these steps:

1. During each epoch, generate the Q-values for each transition by running the agents through the entire episode to ensure temporal relationships are captured
2. Use the generated Q-values and global states for each transition as inputs to compute the Q_{total} values
3. Calculate the temporal difference error for each transition
4. Randomly sample 32 transitions across all episodes and sum them to generate the overall TD loss

4.3 First Iteration

4.3.1 Training Setup

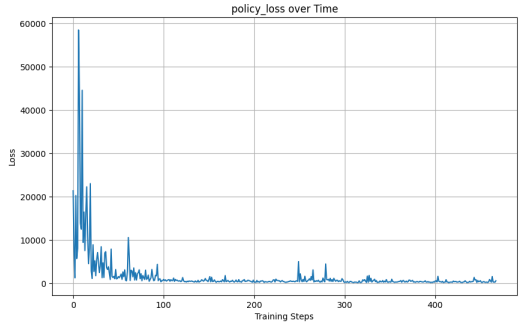
I adopted most of the original parameters with the following modifications:

1. Update interval: Reduced to once every 32 episodes collected to accelerate learning
2. Discount factor for TD error: 0.9 instead of 0.99 in the paper, as I determined that 0.99 may be too large and could potentially cause the model to collapse to a short-term policy
3. Number of training epochs per update: 20 (this parameter was not specified in the original paper)

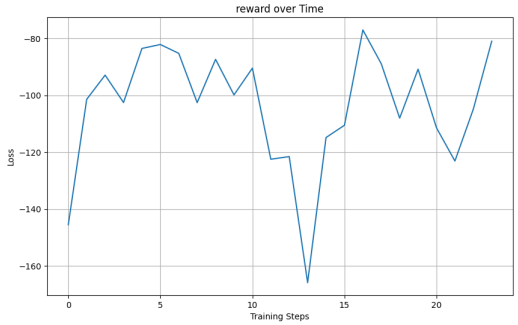
4.3.2 Analysis

Training progressed significantly slower than anticipated, requiring hours to complete just 1,000 timesteps. The following observations were made:

1. Due to the limited number of timesteps, the epsilon values for the agents remained close to 1, effectively making each agent follow an almost entirely exploratory/random policy
2. Consequently, the reward over time showed no signs of improvement
3. The overall TD loss converged despite the data being collected from what were essentially random policies
4. This convergence indicates that the model was collapsing into generating similar Q_{total} values for all states in order to minimize TD error across all transitions



(a) Policy loss over time during the first iteration of training.



(b) Average reward per episode during the first iteration of training.

Figure 2: Data collected during first iteration of training.

4.3.3 Simulation Results

When deploying the trained agents in the environment, random actions were observed, aligning with the analysis above.

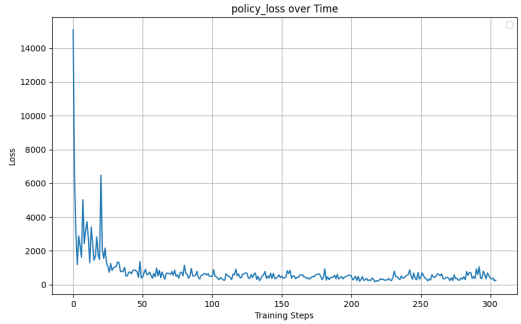
4.4 Second Iteration

4.4.1 Training Setup

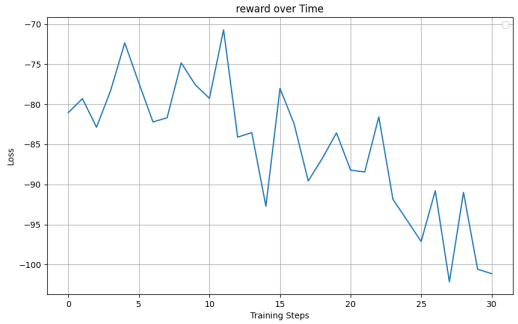
The following modification was implemented:

1. Increased the rate of epsilon reduction by a factor of 10 to accelerate the transition from a random policy to a Q-value exploitation policy

The same data collection approach was maintained as in the first iteration.



(a) Policy loss over time during the second iteration of training.



(b) Average reward per episode during the second iteration of training.

Figure 3: Data collected during second iteration of training.

4.4.2 Analysis

The results reflected the expected effects of the modifications:

1. As epsilon shifted from a purely exploratory policy to a more Q-value-driven exploitation policy, rewards initially decreased, indicating that the agents had not yet learned an effective model of the environment

2. The policy loss exhibited substantially higher variance throughout the training period, suggesting that the model was actively learning from the environment rather than collapsing as observed in the previous iteration

4.4.3 Simulation Results

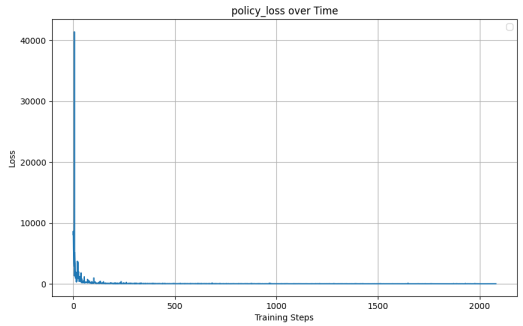
The agents were observed to disperse from one another, though none made active efforts to approach the landmarks. This behavior indicates that the policy network had partially learned aspects of the environment related to agent collision avoidance but had not yet developed strategies for landmark coverage.

4.5 Third Iteration

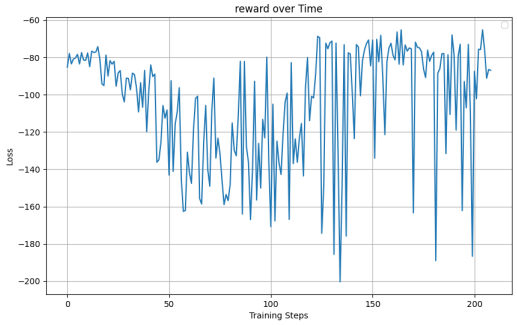
4.5.1 Training Setup

The following modifications were implemented:

1. Reduced the number of training epochs per update from 20 to 10 to slow down model updates
2. Normalized rewards to the range $[-1, 1]$ to stabilize TD error variance and scale
3. Collected additional data to analyze the correlation between agent Q-values and Q_{total}, which is crucial for evaluating whether the environment supports monotonic payoff structure—a key requirement for QMIX to function effectively



(a) Policy loss over time during the third iteration of training.



(b) Average reward per episode during the third iteration of training. The reward here is not normalized for better comparision.

Figure 4: Data collected during third iteration of training.

4.5.2 Analysis

The collected data revealed significant issues in the training process:

1. Rewards became increasingly unstable, indicating that agents were not learning an effective policy
2. Policy loss converged rapidly to very low values, suggesting a similar policy collapse to that observed in the first iteration
3. Agent Q-value and Q_{total} correlation remained positive throughout the training process and gradually increased to approximately 0.9, indicating that monotonicity was successfully maintained

4.5.3 Simulation Results

A major problem was observed in the simulation: agents actively gathered together instead of dispersing, exhibiting behavior opposite to that observed in the previous iteration. Further analysis revealed that the Q_{total} values generated were inversely related to the state rewards—states with more negative rewards were associated with higher Q_{total} values. This finding suggests that QMIX had distorted the policy to maintain monotonicity, resulting in learned behavior contrary to the task requirements.

A critical finding emerged from further analysis of the relationship between actual environmental rewards and the predicted Q_{total} values generated by the mixing network. As shown in Figure 7, there exists a statistically significant negative correlation (Pearson's $r = -0.405$, $p < 0.001$) between these values. This negative correlation directly contradicts the fundamental assumption of reinforcement learning that higher Q-values should correspond to higher expected rewards. The scatter plot reveals that as the actual rewards increase (become less negative), the predicted Q_{total} values consistently decrease, with the trend line having a clear negative slope.

This inverse relationship explains the counterintuitive behavior observed in the agents during simulation. Since the QMIX architecture enforces monotonicity between individual agent Q-values and Q_{total}, and actions are selected to maximize these Q-values, the agents are effectively being trained to pursue states with lower rewards. The mixing network has learned to assign higher Q_{total} values to states with more negative rewards, thereby incentivizing agents to gather together despite the collision penalties.

This misalignment likely stems from one of two sources: either (1) the environment dynamics in the simple spread scenario do not naturally support monotonic value function factorization as required by QMIX, or (2) the reward function design fails to properly align individual agent incentives with the joint team objective. This finding motivated the reward function modifications implemented in the fourth iteration.

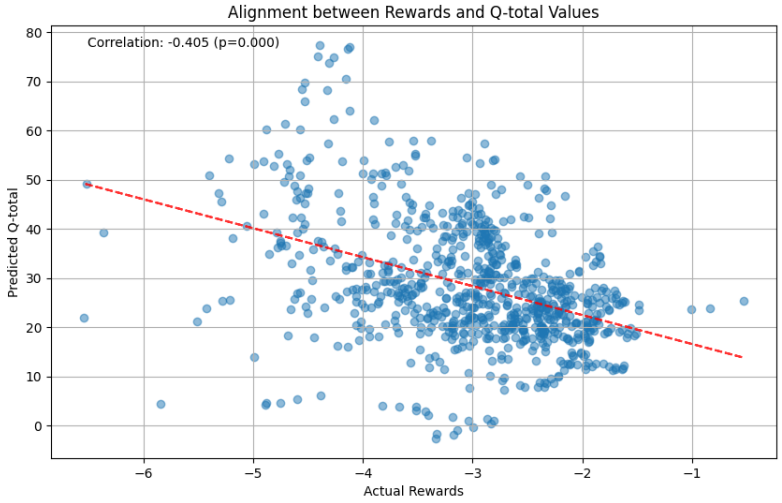


Figure 5: Scatter plot showing negative correlation between environmental rewards and predicted Q_{total} values, with Pearson's $r = -0.405$ ($p < 0.001$).

4.6 Fourth Iteration

4.6.1 Training Setup

The following modifications were implemented:

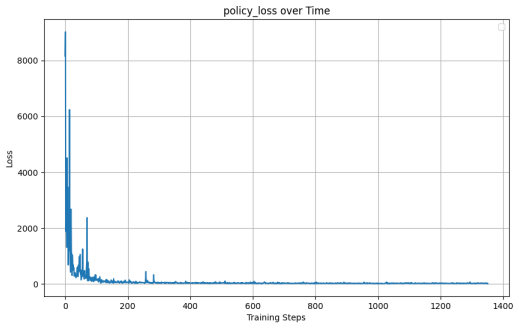
1. Reduced replay buffer size to retain only the most recent 500 episodes
2. Computed Q_{total} only once for each batch of complete episodes
3. Limited the number of updates to 5
4. Reverted the discount factor to 0.9

Additionally, the reward function was modified to better align with the task requirements. The original reward function had two significant limitations:

1. It did not reward agents for approaching landmarks (the sum of minimum distances decreases as agents move closer to landmarks)
2. It did not adequately incentivize agents to maintain distance from each other (only punishing collisions)

The new reward function incorporated:

1. +1 for each agent-landmark overlap and -1 for each agent-agent collision
2. A continuous reward in the range $[0, 1]$ for approaching landmarks:
   - When the minimum distance between a landmark and an agent is less than 1: $\frac{1}{3} \times (1 - \text{minimum\_distance})$
3. A continuous penalty in the range $[0, 1]$ for agents approaching each other:
   - When the minimum distance between two agents is less than 1: $\frac{1}{3} \times (1 - \text{minimum\_distance})$



(a) Policy loss over time during the fourth iteration of training.

(b) Average reward per episode during the fourth iteration of training.

Figure 6: Data collected during fourth iteration of training.

4.6.2 Analysis

Additional data was collected to analyze the correlation between Q_{total} and the rewards at each transition. The analysis revealed no correlation, indicating that QMIX was not effectively learning the environment dynamics or the revised reward structure.

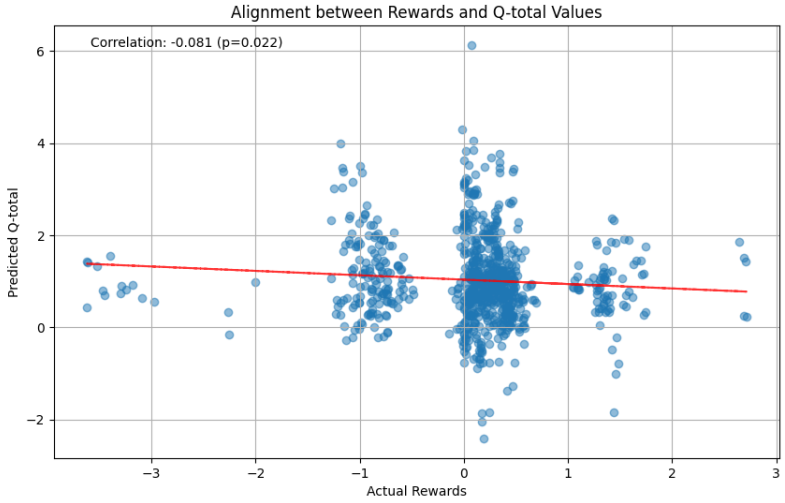


Figure 7: Scatter plot showing negative correlation between environmental rewards and predicted Q_{total} values, with Pearson's $r = -0.081$ ($p < 0.001$).

5 Conclusion

This project implemented and evaluated QMIX, a state-of-the-art centralized training with decentralized execution (CTDE) algorithm, applied to the Multi-Agent Particle Environment (MPE) simple spread scenario. Through four iterations of experimentation, several important insights were gained about both the algorithm's capabilities and the challenges inherent in applying value function factorization to cooperative multi-agent tasks.

5.1 Key Findings

The implementation journey revealed several critical aspects of QMIX's performance and limitations:

1. **Monotonicity constraints can distort policy learning**: While QMIX successfully maintained the monotonicity constraint between individual agent Q-values and the joint Q_{total} (with correlation reaching 0.9), this enforcement came at a significant cost. The negative correlation ($r = -0.405$) between Q_{total} and actual environmental rewards indicated that QMIX was effectively incentivizing agents to pursue states with lower rewards, directly contradicting the fundamental principle that higher Q-values should correspond to higher expected returns.

2. **Reward function design is critical**: The original reward function for the simple spread environment proved inadequate for guiding effective agent behavior. It failed to reward landmark approach and primarily punished collisions without incentivizing proper spacing. The fourth iteration's modified reward function attempted to address these limitations by introducing explicit rewards for landmark proximity and maintaining agent separation.

3. **Training dynamics require careful tuning**: The first iteration demonstrated how epsilon-greedy exploration schedules significantly impact learning progression. With epsilon values remaining near 1 for too long, agents predominantly followed random policies, leading to policy collapse where the model generated similar Q_{total} values for all states to minimize temporal difference error.

4. **Environment compatibility with algorithmic assumptions**: The negative correlation between rewards and Q_{total} suggests that the simple spread environment may not naturally support monotonic value function factorization as required by QMIX. This raises important questions about the applicability of QMIX to certain types of cooperative multi-agent tasks.

5.2 Limitations and Future Work

Several limitations were encountered during this implementation that suggest directions for future research:

1. **Computational constraints**: Resource limitations necessitated reducing network sizes and modifying training parameters, potentially limiting the algorithm's expressive capacity compared to the original paper implementation.

2. **Alternative value function factorization approaches**: Given the challenges encountered with QMIX's monotonicity constraint, exploring algorithms like QTRAN or Weighted QMIX that relax this constraint could yield better performance in environments where monotonicity may be too restrictive.

3. **Reward shaping techniques**: More sophisticated reward shaping beyond the simple modifications attempted in the fourth iteration could better align individual agent incentives with the team objective of covering landmarks while avoiding collisions.

4. **Extended training duration**: The limited training timesteps in this implementation may have been insufficient for convergence to optimal policies, particularly given the complexity of coordination required in the simple spread environment.

5.3 Final Assessment

While QMIX offers theoretical advantages through its ability to represent a larger class of joint action-value functions compared to simpler alternatives like VDN, this implementation revealed practical challenges in applying the algorithm to the MPE simple spread scenario. The enforced monotonicity constraint, while mathematically elegant and beneficial for decentralized execution, can conflict with environmental reward structures that do not naturally support such factorization.

This work demonstrates the importance of alignment between algorithmic assumptions and environment dynamics in multi-agent reinforcement learning. Despite the challenges encountered, the experiments provided valuable insights into both the strengths and limitations of value function factorization approaches to cooperative multi-agent tasks. These findings contribute to the broader understanding of when and how to effectively apply CTDE algorithms in complex coordination scenarios.